



中國海洋大學

OCEAN UNIVERSITY OF CHINA

Summer 2026 Semester

Robot Integration Group Project 1 Report

Experiment 2: Smartphone Camera Intrinsic Calibration

OpenCV Chessboard Calibration of a Smartphone Camera

NAME: Jiasheng Gu

STUDENT ID: 24020036020

Experiment topic: Intrinsic and distortion calibration using a 9×6 checkerboard

Abstract

This experiment calibrates the intrinsic parameters and lens distortion of a smartphone camera using an A4 checkerboard containing 9×6 squares, 8×5 inner corners, and a 30 mm square size. OpenCV is used for checkerboard detection, sub-pixel refinement, and nonlinear calibration with `calibrateCamera`. The original dataset contains 30 photographs with mixed resolutions and orientations. To keep one consistent pixel coordinate system, the 16 photographs at 960×1280 were selected as the candidate group: 14 images were detected successfully, giving an image-level detection success rate of 87.50%. After rejecting two views whose individual reprojection errors exceeded 1.5 pixels, 12 views were used for the final solution. The final OpenCV RMS reprojection error is 0.893687 pixels, the mean per-view error is 0.876530 pixels, and leave-one-out validation gives a mean error of 0.957206 pixels. The result is suitable for image undistortion and visual measurement at the same resolution, orientation, lens, and zoom setting. This detection rate is not a substitute for a physical millimetre-level robot positioning accuracy test.

Keywords: smartphone camera; intrinsic parameters; checkerboard; OpenCV; distortion correction; reprojection error

Contents

Abstract	2
Contents	3
1 Experimental Requirements and Objectives	4
1.1 Objectives	4
1.2 Calibration Board and Conditions	4
2 Calibration Principle and Mathematical Model	4
2.1 World Coordinates of Checkerboard Points	4
2.2 Pinhole Camera Model	4
2.3 Lens Distortion Model	5
2.4 Optimization Objective and Error	5
3 Experimental Procedure and OpenCV Implementation	6
3.1 Procedure	6
3.2 Core OpenCV Program	6
4 Dataset and Experimental Results	7
4.1 Dataset Composition	7
4.2 Corner Detection Results	8
4.3 Intrinsic and Distortion Results	9
5 Calculation Process and Quality Evaluation	11
5.1 From Corners to Intrinsic Parameters	11
5.2 Plausibility Checks	11
5.3 Undistortion Verification	11
5.4 Limitations	11
6 Problems and Improvements	11
6.1 Mixed Resolution	11
6.2 Failed Corner Detection	11
6.3 Further Accuracy Improvements	11
7 Personal Summary	12
References	12
Appendix A Output Files	13
Appendix B Reading the Calibration File	13

1 Experimental Requirements and Objectives

1.1 Objectives

The objective of camera calibration is to estimate the intrinsic matrix and lens distortion parameters from known three-dimensional points and their image observations. The intrinsic matrix describes focal lengths, the principal point, and the pixel coordinate scale. The distortion parameters describe the deviation of the real lens from the ideal pinhole model. The experiment requires smartphone images of a printed checkerboard, automatic corner detection with OpenCV, numerical parameter estimation, and verification using visual outputs and quantitative errors.

1.2 Calibration Board and Conditions

Item	Setting in this experiment	Description
Checkerboard squares	9×6	Number of printed black/white squares
OpenCV inner corners	8×5 = 40	Intersections between adjacent squares
Square size	30 mm	Unit used for world coordinates
Image source	Smartphone camera	One lens and one imaging mode
Candidate dataset	30 JPG files	Extracted from archive.zip
Final image size	960×1280	Portrait, unscaled candidate group

The checkerboard should be flat and fixed. Printing must use 100% scale rather than “fit to page”. During acquisition, the board should be moved to different distances, positions, and tilt angles so that corners cover the image centre, borders, and corners. Blur, reflections, overexposure, and occlusion should be avoided. Different smartphone lenses have different intrinsic parameters and must be calibrated separately.

2 Calibration Principle and Mathematical Model

2.1 World Coordinates of Checkerboard Points

The checkerboard plane is defined as $Z=0$ in the world coordinate system. With 8 inner corners horizontally and 5 vertically, the point at row i and column j is:

$$P_{ij} = \begin{bmatrix} 30j \\ 30i \\ 0 \end{bmatrix} \text{ mm}, \quad i = 0, \dots, 4, \quad j = 0, \dots, 7$$

The absolute square-size unit changes the translation units between the camera and the board, but does not change the focal length and principal point expressed in pixels.

2.2 Pinhole Camera Model

The ideal projection is written in homogeneous coordinates as:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Here K is the intrinsic matrix, R and t are the rotation and translation of the board relative to the camera, and s is the homogeneous scale. The no-skew form used in this experiment is:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

2.3 Lens Distortion Model

The normalized image coordinates and radius are:

$$x = \frac{u - c_x}{f_x}, \quad y = \frac{v - c_y}{f_y}, \quad r^2 = x^2 + y^2$$

The five-parameter radial/tangential model is:

$$\begin{aligned} x_d &= x (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2 (r^2 + 2x^2) \\ y_d &= y (1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1 (r^2 + 2y^2) + 2p_2 xy \end{aligned}$$

OpenCV stores these coefficients in the order [k_1, k_2, p_1, p_2, k_3].

2.4 Optimization Objective and Error

For detected point p_{ij} and its model projection $\hat{p}_{ij}$, calibration minimizes the following objective:

$$E = \sum_{i,j} \|p_{ij} - \hat{p}_{ij}\|^2$$

OpenCV reports the square root of the mean squared residual as RMS. This report also calculates each-view error and a leave-one-out validation error.

3 Experimental Procedure and OpenCV Implementation

3.1 Procedure

1. Print the checkerboard at 100% scale and verify the 30 mm square size.
2. Capture the board with one smartphone lens at different distances, positions, and tilt angles.
3. Group images by resolution and orientation so their pixel coordinate systems are consistent.
4. Detect the 8×5 inner corners with findChessboardCornersSB, falling back to findChessboardCorners.
5. Build 40 corresponding 3-D world points and 2-D image points for every successful view.
6. Call calibrateCamera to estimate K, distortion, extrinsics, and RMS.
7. Reject obvious outlier views, recalibrate, and save YAML/JSON/error reports.
8. Generate undistorted images with undistort and visually inspect straight checkerboard lines.

3.2 Core OpenCV Program

```
import cv2
import numpy as np
from pathlib import Path

BOARD = (8, 5)          # inner corners, not square count
SQUARE_MM = 30.0

def detect(gray):
    if hasattr(cv2, "findChessboardCornersSB"):
        flags = (cv2.CALIB_CB_NORMALIZE_IMAGE |
                  getattr(cv2, "CALIB_CB_EXHAUSTIVE", 0) |
                  getattr(cv2, "CALIB_CB_ACCURACY", 0))
        ok, corners = cv2.findChessboardCornersSB(gray, BOARD, flags)
        if ok:
            return ok, corners.astype(np.float32)
    flags = cv2.CALIB_CB_ADAPTIVE_THRESH | cv2.CALIB_CB_NORMALIZE_IMAGE
    ok, corners = cv2.findChessboardCorners(gray, BOARD, flags)
    if ok:
        criteria = (cv2.TERM_CRITERIA_EPS |
                    cv2.TERM_CRITERIA_MAX_ITER, 50, 0.001)
        corners = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)
    return ok, corners

obj = np.zeros((BOARD[0] * BOARD[1], 3), np.float32)
obj[:, :2] = np.mgrid[0:BOARD[0], 0:BOARD[1]].T.reshape(-1, 2)
obj[:, :2] *= SQUARE_MM

object_points, image_points = [], []
image_size = None
for filename in sorted(Path("calibration_images").glob("*.jpg")):
    image = cv2.imread(str(filename))
    if image is None:
        continue
    size = (image.shape[1], image.shape[0])
    if image_size is None:
        image_size = size
    if size != image_size:
        continue
    ok, corners = detect(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY))
    if ok:
        object_points.append(obj.copy())
        image_points.append(corners)

rms, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    object_points, image_points, image_size, None, None)
print("K =\n", K)
print("dist =", dist.ravel())
print("RMS =", rms)
```

The full program additionally handles non-ASCII paths, saves corner-check images, rejects outlier views, writes YAML/JSON, and produces undistorted examples. The key setting is BOARD=(8,5): the printed board has 9×6 squares but only 8×5 inner corners.

4 Dataset and Experimental Results

4.1 Dataset Composition

Resolution	Count	Treatment
960×1280	16	Final candidate group, original size
1280×1707	6	Not mixed into this run
1702×1276	3	Landscape, not mixed
1276×958	1	Landscape, not mixed
1024×1366	4	Not mixed into this run
Total	30	Original archive dataset

Figure 1 shows all 30 archived images. Because the archive contains different resolutions and portrait/landscape orientations, mixing them directly would assign one intrinsic matrix to multiple pixel coordinate systems. The 960×1280 group was therefore used for the reported calibration.

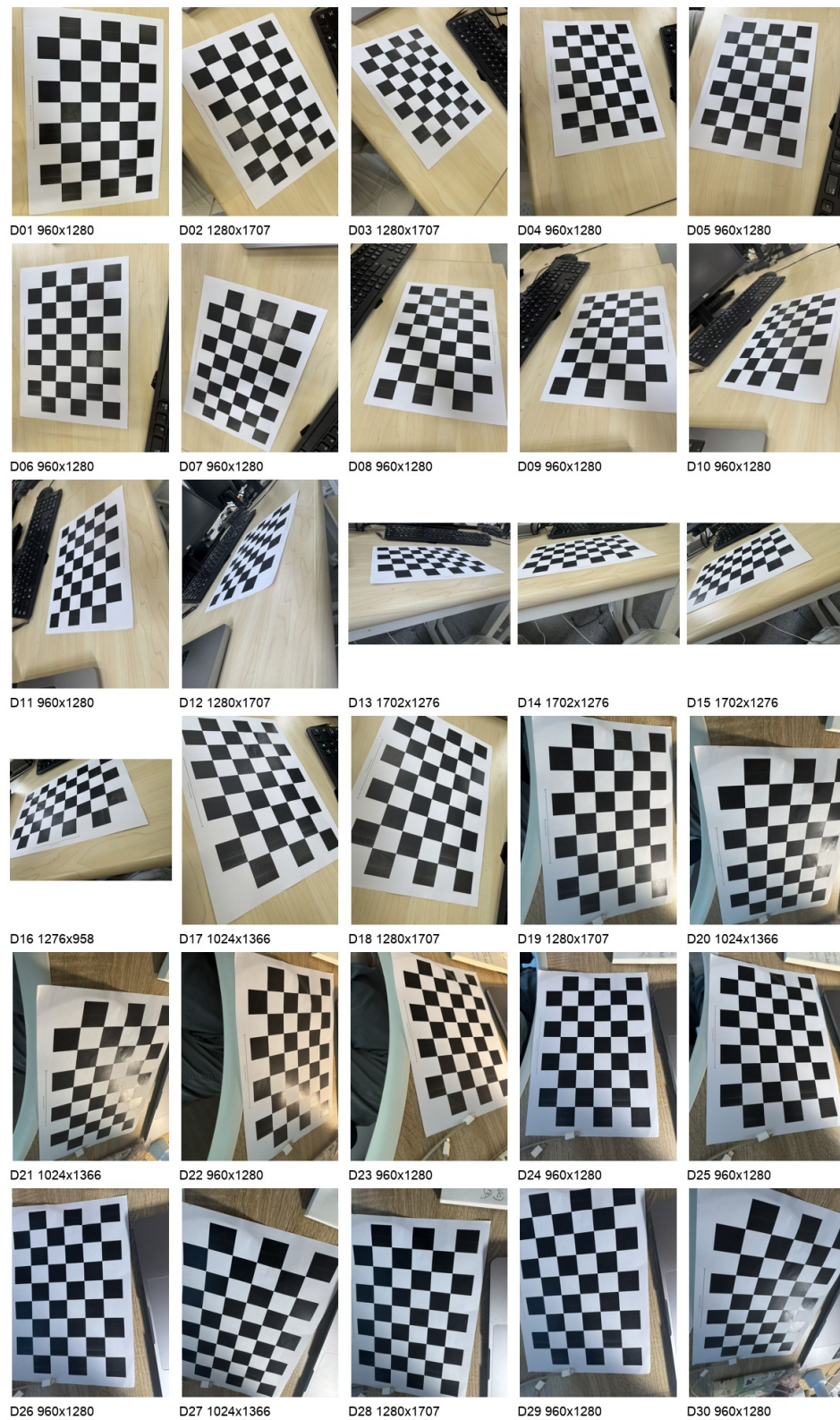


Figure 1. Smartphone checkerboard dataset overview (30 images; resolution labels)

4.2 Corner Detection Results

Fourteen of the 16 candidate images contained a complete set of 8×5 inner corners. Two views failed because the board was incomplete or the perspective/sharpness was insufficient. The image-level detection success rate is:

$$\text{Detection rate} = \frac{14}{16} \times 100\% = 87.50\%$$



Figure 2. Detected checkerboard corners (40 inner corners per successful image)

The views cover different board positions and tilt angles. However, slight paper warping and messaging-app compression introduce additional corner-localization noise.

4.3 Intrinsic and Distortion Results

After removing two views with individual reprojection errors above 1.5 pixels, 12 images were used for the final calibration at 960×1280. The intrinsic matrix is:

$$K = \begin{bmatrix} 919.92997148 & 0 & 473.34004439 \\ 0 & 919.07542347 & 639.34287186 \\ 0 & 0 & 1 \end{bmatrix}$$

Thus $f_x=919.92997$ px, $f_y=919.07542$ px, and the principal point is $(c_x, c_y)=(473.34004, 639.34287)$ px. The principal point is close to the image centre (480,640), while f_x and f_y differ by approximately 0.09%. The distortion vector is shown below in the OpenCV order $[k_1, k_2, p_1, p_2, k_3]$.

$$\mathbf{d} = [k_1, k_2, p_1, p_2, k_3] = [0.282606680, -1.19331305, -0.00141411804, -0.00587499852, 1.49297782]$$

Metric	Result	Meaning
OpenCV RMS	0.893687 px	Overall RMS from calibrateCamera
Mean per-view error	0.876530 px	Mean of 12 final views
Leave-one-out mean	0.957206 px	One image held out per refit
Leave-one-out maximum	1.309898 px	Largest validation error
Candidate detection rate	87.50%	14 of 16 complete detections

A mean reprojection error below one pixel indicates a usable pixel-domain fit. It must not be interpreted as millimetre-level robot positioning accuracy; that requires an independent metric or control-point experiment.

5 Calculation Process and Quality Evaluation

5.1 From Corners to Intrinsic Parameters

First, the program generates the same 40 world points for every image and pairs them with the detected two-dimensional corners. Second, OpenCV estimates an initial pose for every planar board using homographies; multiple board poses constrain the common intrinsic matrix. Third, nonlinear optimization jointly refines K , distortion, and each image pose to minimize the total squared reprojection residual.

For image i , with extrinsics R_i and t_i , the predicted point is:

$$\hat{p}_{ij} = \text{project}(K, \mathbf{d}, R_i, t_i, P_j)$$

The Euclidean distance between the detected and predicted point is computed for all 40 corners. The residuals are combined into RMS. The two views above 1.5 pixels were then rejected and calibration was repeated.

5.2 Plausibility Checks

The principal point is close to the image centre, indicating consistent image-size and corner coordinates. The similar f_x and f_y values agree with nearly square smartphone pixels. The small tangential terms indicate that radial distortion explains most of the lens deviation. The leave-one-out mean error increases by only about 0.081 pixels over the training mean, so the solution is not dominated by one image.

5.3 Undistortion Verification

The program uses `getOptimalNewCameraMatrix` to choose a useful corrected field of view and then calls `undistort`. Straightening of the checkerboard rows and columns provides a qualitative check. Example corrected images are included in the result package.

5.4 Limitations

The archive contains compressed images, mixed resolutions, a paper board that is not perfectly rigid, and two failed detections. The result is appropriate for a standard pixel-level calibration experiment, but should not be presented as a high-precision 3-D measurement calibration. For a higher-quality repeat, use original JPG/PNG files, one lens and resolution, 20–30 sharp poses, and a board fixed to a rigid flat plate.

6 Problems and Improvements

6.1 Mixed Resolution

Focal length and principal point are expressed in pixels, so resizing, cropping, or rotating changes their numerical representation. The program reports incompatible resolutions and uses the largest consistent group instead of silently mixing them.

6.2 Failed Corner Detection

Failure is usually caused by an incomplete board, blur, reflections, excessive tilt, or occlusion. The program first uses the robust SB detector and falls back to the classical detector; failed images are never added to calibration.

6.3 Further Accuracy Improvements

Fix the board to a rigid plate, keep the lens, resolution, orientation, and zoom fixed, capture 20–30 poses that cover the image centre and edges, and aim for a mean reprojection error around 0.5 pixels. For robot use, continue with camera extrinsic, table-plane, or hand-eye calibration; camera intrinsics alone do not yield robot coordinates.

7 Personal Summary

This experiment completed the workflow from checkerboard printing and smartphone acquisition to corner detection, intrinsic estimation, distortion modeling, and undistortion verification. The data processing clarified the relationship between camera parameters and image resolution and reinforced that a 9×6 square board corresponds to 8×5 OpenCV inner corners. RMS, per-view error, and leave-one-out validation were used together to assess reliability. The result is usable at the reported image size, while compression, resolution mixing, and paper warping limit the achievable precision.

References

- [1] OpenCV Documentation. Camera Calibration and 3D Reconstruction. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [2] Z. Zhang. A Flexible New Technique for Camera Calibration. IEEE Transactions on Pattern Analysis and Machine Intelligence, 22(11):1330–1334, 2000.
- [3] R. Hartley and A. Zisserman. Multiple View Geometry in Computer Vision. Cambridge University Press, 2nd ed., 2004.
- [4] Experimental dataset: archive.zip, smartphone checkerboard photographs, 2026-09-17.

Appendix A Output Files

File	Purpose
camera_calibration.yaml	Camera matrix, distortion coefficients, and image size for OpenCV/ROS
camera_calibration.json	Human-readable parameters, image list, and per-view errors
calibration_report.txt	RMS, per-view errors, and image-selection record
calibration_accuracy.json	Detection rate and leave-one-out metrics
detected_corners/	Corner-check images with 40 detected points
undistorted_images/	Images corrected using the calibration parameters

Appendix B Reading the Calibration File

```
import cv2

fs = cv2.FileStorage("camera_calibration.yaml", cv2.FILE_STORAGE_READ)
K = fs.getNode("camera_matrix").mat()
dist = fs.getNode("distortion_coefficients").mat()
fs.release()

image = cv2.imread("test.jpg")
undistorted = cv2.undistort(image, K, dist)
cv2.imwrite("test_undistorted.jpg", undistorted)
```